

Credit Score Prediction and Analysis

Using Logistic Regression: A Statistical Study on the German Credit Risk Dataset

A Project Report on Exploratory Data Analysis,
Statistical Inference, and Predictive Modeling of Credit Risk

Submitted by:
Soumya Chanda

Date:
June 22, 2026

Dataset: Statlog (German Credit Data), UCI Machine Learning
Repository

Method: Binary Logistic Regression, full model compared against a
reduced model

Tools: Python (pandas, statsmodels, scikit-learn, seaborn)

June 22, 2026

Contents

Abstract	2
1 Introduction	3
1.1 What this report tries to do	3
1.2 How the report is organized	3
2 Dataset Description	3
2.1 Cost Matrix	4
3 Exploratory Data Analysis	4
3.1 Data Quality	4
3.2 Target Variable	5
3.3 Univariate Analysis	5
3.3.1 Numerical Features	5
3.4 Categorical Features vs. Target	7
3.5 Correlation Analysis	9
3.6 Statistical Significance Testing	9
3.7 Multicollinearity (Numerical Features)	10
3.8 Summary of the EDA	10
4 Modeling Methodology	11
4.1 Why Logistic Regression	11
4.2 Preparing the Data	11
4.2.1 Target Encoding	11
4.2.2 One-Hot Encoding	11
4.2.3 Train-Test Split	11
4.2.4 Scaling	11
4.3 Checking Multicollinearity Again on the Full Matrix	11
5 Full Logistic Regression Model	12
6 Reduced Model via Backward Elimination	12
7 Full Model vs. Reduced Model: Is the Smaller Model Good Enough?	13
8 How Well Do They Actually Predict?	14
9 Choosing a Threshold That Reflects the Real Cost	15
10 Final Model and What It Means	16
10.1 Which Model Should Actually Be Used	16
10.2 Reading the Coefficients	16
11 Conclusion and Limitations	17
11.1 Main Takeaways	17
11.2 Limitations	17

Appendix A: Full Backward Elimination Log	19
--	-----------

Appendix B: Data and Tools	20
-----------------------------------	-----------

Abstract

This report works through a complete statistical pipeline for predicting credit risk with Logistic Regression, using the Statlog (German Credit Data) dataset from the UCI Machine Learning Repository. The first part is an exploratory analysis of the 1,000 applicant records: checking data quality, looking at how the target variable and each predictor are distributed, and then running formal hypothesis tests (Chi-Square and Mann–Whitney U) to see which of the twenty available attributes actually look related to credit risk before any model is built. The second part fits a full Logistic Regression model with all 48 encoded predictors and compares it against a smaller model obtained by backward elimination, using a Likelihood Ratio Test along with AIC and BIC. Both models are then evaluated on a held-out test set using F1-score and ROC-AUC, and a final threshold is chosen using the dataset’s own asymmetric cost matrix instead of just defaulting to 0.5. The reduced model, with 17 predictors left after elimination, turns out to be statistically just as good as the full model and is far easier to explain, though it does give up a bit of recall on bad-risk applicants at the default threshold – a trade-off this report tries to be upfront about rather than hide. The coefficients of the final model are interpreted as odds ratios so that the conclusions can be stated in plain terms, not just as numbers in a table.

Keywords: Logistic Regression, Credit Risk, Likelihood Ratio Test, Backward Elimination, ROC-AUC, Cost-Sensitive Classification.

1 Introduction

Deciding whether a loan applicant is a good bet or a bad one is a problem banks have been dealing with for a long time, and it is also a fairly natural place to apply statistics, since the decision essentially comes down to estimating a probability from a handful of observed characteristics. In this project I used **Logistic Regression** to model this decision rather than a more complex machine learning algorithm, mainly because the whole point of this exercise was to practice the underlying statistics – maximum likelihood estimation, the logit link, hypothesis tests on individual coefficients – and not just to get the best possible accuracy number. A random forest or a gradient boosted model might predict slightly better, but it would not tell me *why* an applicant was flagged as risky, and that interpretability is really the point of this report.

1.1 What this report tries to do

Broadly, the work is split into four parts:

- (i) Explore the dataset properly before touching any model – look at the target variable, the shape of each predictor, and how they relate to each other.
- (ii) Fit a full Logistic Regression model on every available predictor and use hypothesis testing to figure out which variables are actually contributing something once everything else is accounted for, and which ones are not.
- (iii) Build a reduced model through backward elimination and test, formally, whether dropping those variables was actually justified (Likelihood Ratio Test, AIC, BIC), rather than just assuming a smaller model is fine.
- (iv) Check how well both models predict on data they have not seen, using F1-score and ROC-AUC, and pick a decision threshold that reflects the real cost of getting a prediction wrong in this context.

1.2 How the report is organized

Section 2 describes the dataset. Section 3 is the exploratory analysis. Section 4 covers how the data was prepared for modeling. Sections 5 through 7 build the full model, the reduced model, and compare the two formally. Section 8 looks at predictive performance, Section 9 deals with threshold selection, Section 10 interprets the final model, and Section 11 wraps up with the main conclusions and a few honest limitations.

2 Dataset Description

The dataset used here is the **Statlog (German Credit Data)** set, originally put together by Prof. Hans Hofmann at the Universität Hamburg and now hosted on the UCI Machine Learning Repository. It has records for 1,000 loan applicants, each with 20 predictor attributes (7 numerical, 13 categorical) and a target column, `credit_risk`, labelling each applicant as **good** or **bad**.

The raw file stores categorical values as codes (**A11**, **A34**, and so on), which are not very readable on their own, so before doing any analysis these were mapped back to descriptive labels using the dataset’s documentation. Table 1 lists every column used in this study.

Table 1: Data dictionary for the German Credit Risk dataset.

#	Column	Type	Description
1	checking_account_status	Categorical	Status/balance of existing checking account
2	duration_months	Numerical	Duration of the credit (loan) in months
3	credit_history	Categorical	Applicant's credit repayment history
4	purpose	Categorical	Purpose of the loan
5	credit_amount	Numerical	Loan amount requested (DM)
6	savings_account	Categorical	Balance in savings account/bonds
7	employment_since	Categorical	Duration in current employment
8	installment_rate_pct	Numerical	Installment rate (% of disposable income)
9	personal_status_sex	Categorical	Combined marital status and sex
10	other_debtors_guarantors	Categorical	Presence of co-applicants/guarantors
11	present_residence_since	Numerical	Years at current residence
12	property	Categorical	Type of property owned
13	age_years	Numerical	Age of applicant (years)
14	other_installment_plans	Categorical	Other installment plans held
15	housing	Categorical	Housing situation
16	existing_credits_count	Numerical	Number of existing credits at this bank
17	job	Categorical	Job/skill classification
18	num_dependents	Numerical	Number of financial dependents
19	telephone	Categorical	Telephone registered under applicant's name
20	foreign_worker	Categorical	Whether applicant is a foreign worker
21	credit_risk	Target	Credit risk: good or bad

2.1 Cost Matrix

One thing that is easy to miss on a first read of the documentation is that the dataset comes with its own cost matrix. Calling a genuinely bad-risk applicant *good* is treated as five times worse than calling a good applicant *bad* (Table 2). This matters a lot later on, in Section 9, when it comes to choosing a threshold – the usual 0.5 cutoff quietly assumes both kinds of mistakes are equally bad, and here they clearly are not.

Table 2: Asymmetric cost matrix specified by the dataset documentation.

Actual \ Predicted	Good	Bad
Good	0	1
Bad	5	0

3 Exploratory Data Analysis

3.1 Data Quality

The good news first: there are no missing values and no duplicate rows anywhere in the 1,000 records, and every categorical column has a clean, finite set of category labels with no typos or odd entries. So there was nothing to clean up before getting into the actual analysis, which is not always the case with real-world data.

3.2 Target Variable

700 of the 1,000 applicants (70%) are good credit risk and 300 (30%) are bad (Figure 1). That is a moderate imbalance, not extreme, but enough that accuracy alone would be a poor way to judge a model here – a model that just predicts "good" every single time would already be 70% accurate without learning anything useful. That is why precision, recall, F1-score, and ROC-AUC are used later in Section 8 instead of leaning on accuracy, and it is also part of why the cost matrix in Section 9 matters.

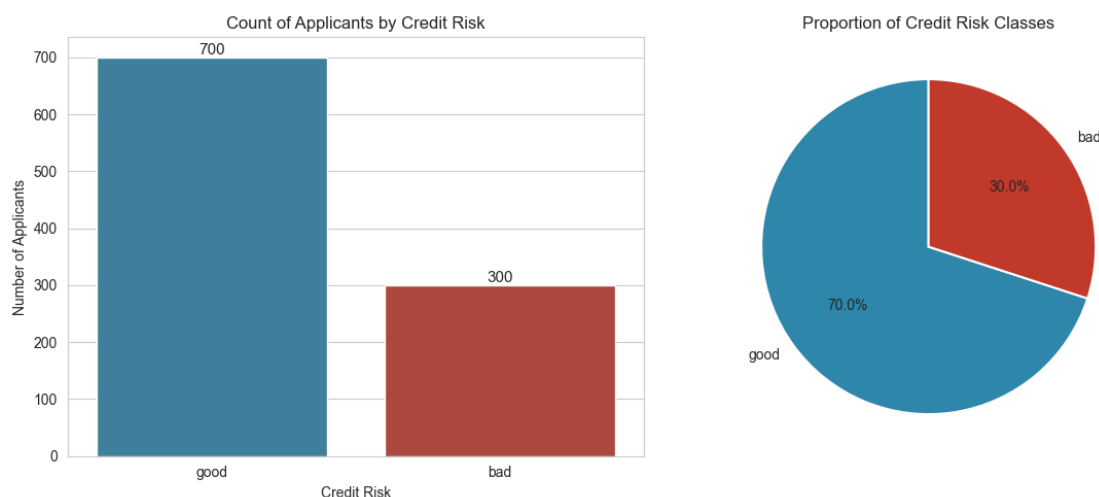


Figure 1: Distribution of the target variable `credit_risk`.

3.3 Univariate Analysis

3.3.1 Numerical Features

Table 3 shows basic summary statistics for the seven numerical columns. `duration_months`, `credit_amount`, `age_years`, `existing_credits_count`, and `num_dependents` are all right-skewed to some degree, while `installment_rate_pct` and `present_residence_since` only take values 1 through 4, so they behave more like ordered categories than continuous measurements. The skew in `credit_amount` in particular (skewness ≈ 1.95) is pretty visible in both the histograms (Figure 2) and the boxplots (Figure 3) – there is a long tail of large loans pulling the mean well above the median. This is also the reason a non-parametric test was used later instead of a t -test (Section 3.6).

Table 3: Distributional summary of the seven numerical features.

Feature	Mean	Median	Std. Dev.	Skewness	Kurtosis
<code>duration_months</code>	20.90	18.0	12.06	1.09	0.92
<code>credit_amount</code>	3271.26	2319.5	2822.74	1.95	4.29
<code>installment_rate_pct</code>	2.97	3.0	1.12	-0.53	-1.21
<code>present_residence_since</code>	2.84	3.0	1.10	-0.27	-1.38
<code>age_years</code>	35.55	33.0	11.38	1.02	0.60
<code>existing_credits_count</code>	1.41	1.0	0.58	1.27	1.60
<code>num_dependents</code>	1.16	1.0	0.36	1.91	1.65

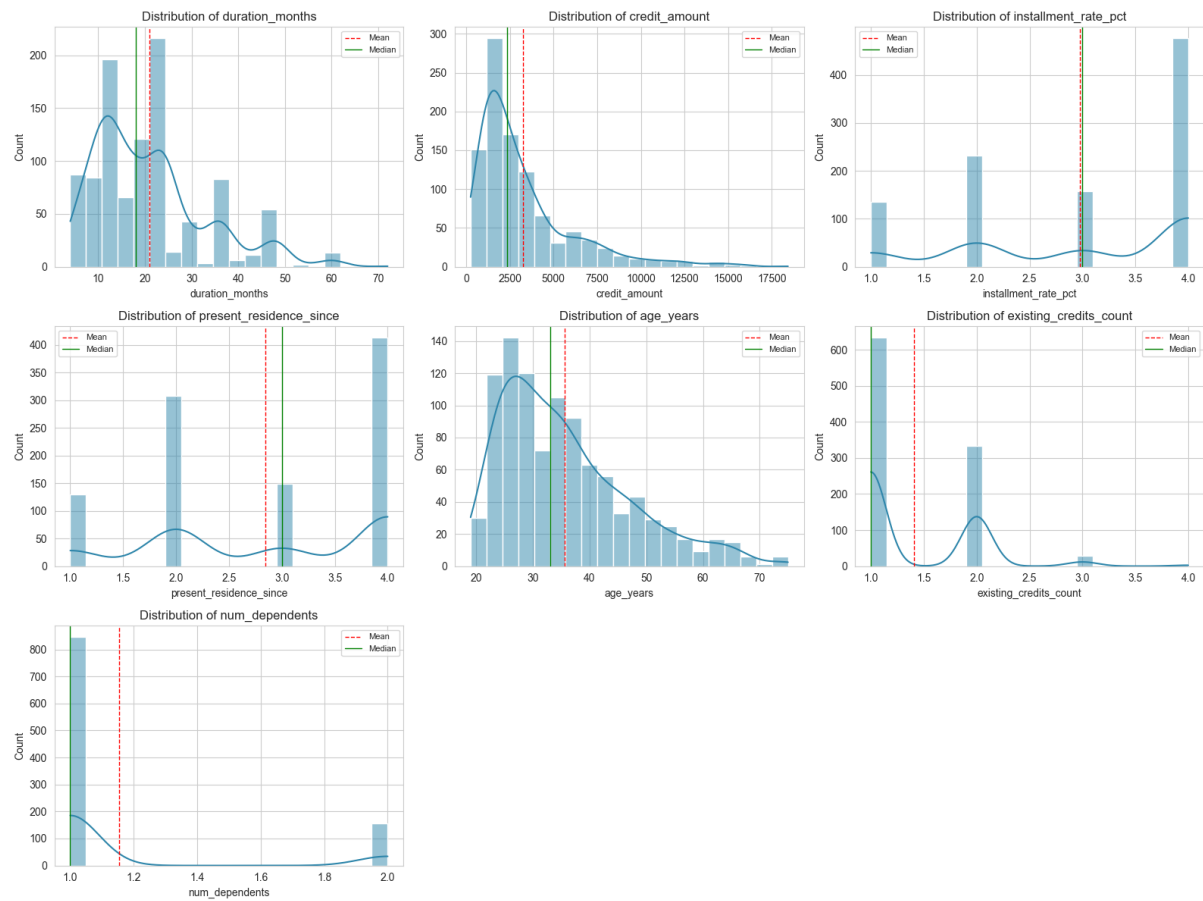


Figure 2: Distributions of the seven numerical features, with mean (red, dashed) and median (green, solid) marked.

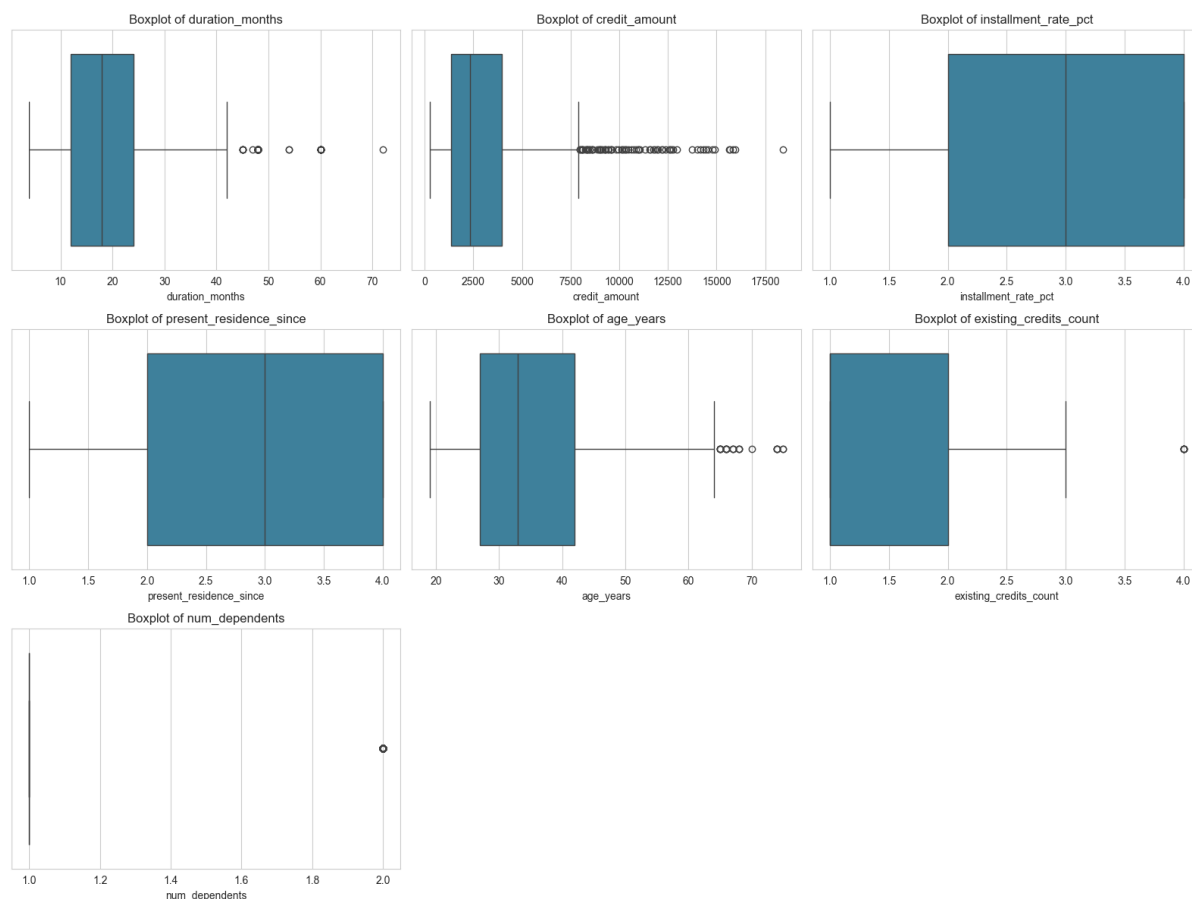


Figure 3: Boxplots of the seven numerical features, used to visually flag potential outliers.

3.4 Categorical Features vs. Target

For each categorical feature, I calculated the percentage of bad-risk applicants within every category and compared it to the overall 30% baseline (Figure 4). `checking_account_status`, `credit_history`, and `savings_account` stand out the most here, which turns out to line up with what shows up as significant in the multivariate model later (Section 5). One pattern worth pointing out: applicants with **no checking account at all** actually have a *lower* bad-risk rate than applicants whose checking balance is below 0 DM. At first that seems backwards, but it is a known quirk of this dataset – not having a checking account does not automatically mean someone is in financial trouble, it can just mean they bank differently.



Figure 4: Percentage of bad-risk applicants within each category, for every categorical feature. The dashed line marks the dataset's overall 30% bad-risk baseline.

3.5 Correlation Analysis

Figure 5 shows the Pearson correlation between the seven numerical features. The only pair worth mentioning is `duration_months` and `credit_amount`, which have a moderate positive correlation – makes sense, since bigger loans tend to be spread out over longer repayment periods. Nothing else really stands out, and nothing is high enough on its own to worry about multicollinearity.

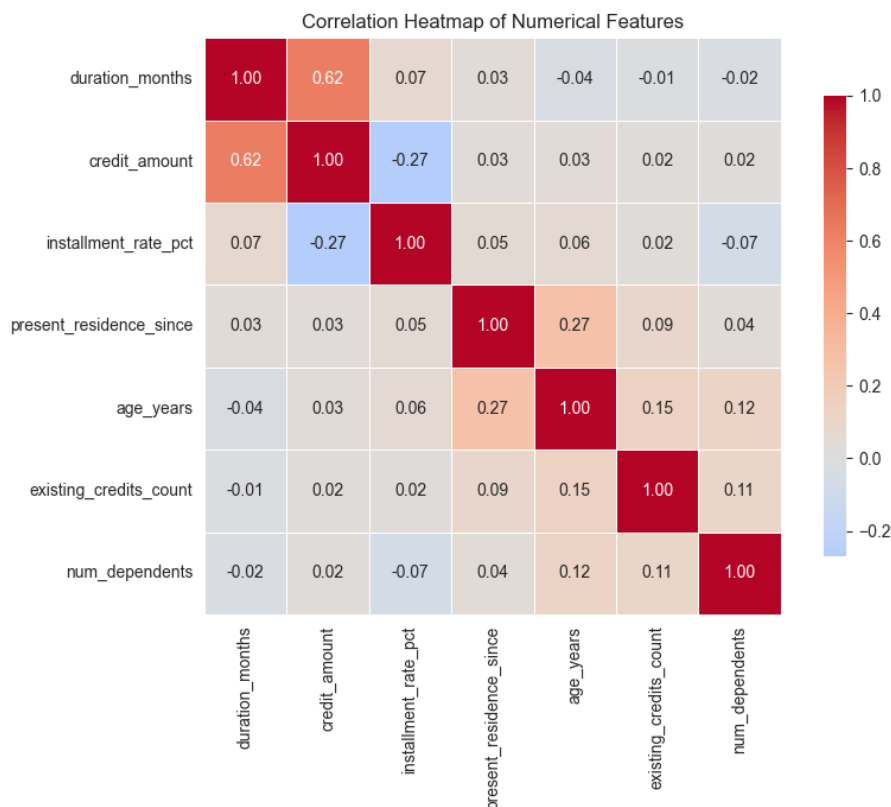


Figure 5: Correlation heatmap of the seven numerical features.

3.6 Statistical Significance Testing

Visual patterns are a useful starting point but they are not a substitute for an actual test, so each predictor was tested formally at $\alpha = 0.05$. For the categorical features, a **Chi-Square Test of Independence** was used against `credit_risk` (Table 4). For the numerical features, I used the **Mann–Whitney U test** rather than a standard *t*-test (Table 5), since Section 3.3.1 already showed several of these features are skewed enough that the normality assumption behind the *t*-test would not really hold.

Table 4: Chi-Square Test of Independence: each categorical feature vs. `credit_risk`.

Feature	χ^2	df	p-value	Significant ($\alpha = 0.05$)
checking_account_status	123.721	3	<0.0001	Yes
credit_history	61.691	4	<0.0001	Yes
savings_account	36.099	4	<0.0001	Yes
property	23.720	3	<0.0001	Yes
purpose	33.356	9	0.0001	Yes
housing	18.200	2	0.0001	Yes
employment_since	18.368	4	0.0010	Yes
other_installment_plans	12.839	2	0.0016	Yes
foreign_worker	5.822	1	0.0158	Yes
personal_status_sex	9.605	3	0.0222	Yes
other_debtors_guarantors	6.645	2	0.0361	Yes
telephone	1.173	1	0.2789	No
job	1.885	3	0.5966	No

Table 5: Mann–Whitney U test: each numerical feature, good vs. bad credit risk groups.

Feature	U statistic	p-value	Significant ($\alpha = 0.05$)
duration_months	77,995.5	<0.0001	Yes
age_years	119,833.0	0.0004	Yes
credit_amount	93,480.0	0.0059	Yes
installment_rate_pct	95,889.5	0.0199	Yes
existing_credits_count	110,272.0	0.1348	No
num_dependents	105,250.0	0.9242	No
present_residence_since	104,680.5	0.9358	No

A couple of patterns are worth noting from these tables. Bad-risk applicants take out loans for longer on average (24.9 vs. 19.2 months) and ask for more money (3,938 vs. 2,985 DM), while good-risk applicants tend to be a little older on average (36.2 vs. 34.0 years). `existing_credits_count`, `num_dependents`, and `present_residence_since` did not come out significant here, which is something to keep in mind once the multivariate model is built in Section 5, since a variable that looks unimportant on its own can sometimes still combine with others in a useful way (or, as turned out to be more common here, the other way around).

3.7 Multicollinearity (Numerical Features)

A quick VIF check on just the numerical columns came back clean, with every value under 2 (`credit_amount` was the highest, at 1.99), so multicollinearity is not an issue among the numerical predictors at this stage. The full design matrix, once the categorical dummies are added in, is checked again in Section 4.3.

3.8 Summary of the EDA

To pull this section together: the data is clean and needed no fixing, the target variable is imbalanced enough to matter for how the model gets evaluated later, a fair number of predictors – both numerical and categorical – show a real statistical relationship with credit risk on their

own, and multicollinearity among the numerical features is not a concern. These points carry straight through into how the modeling was set up, described next.

4 Modeling Methodology

4.1 Why Logistic Regression

Logistic Regression models the log-odds of the outcome $y \in \{0, 1\}$ (here $y = 1$ means bad credit risk) as a linear function of the predictors:

$$\log \left(\frac{P(y = 1 | \mathbf{x})}{1 - P(y = 1 | \mathbf{x})} \right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k. \quad (1)$$

What makes this useful for this report is that each β_j comes with a clean interpretation: e^{β_j} is the odds ratio for a one-unit increase in x_j , holding everything else fixed. Because the coefficients are estimated by maximum likelihood, we also get standard errors, z -statistics, and p -values for free, which is exactly what is needed for the significance testing in Sections 5 through 7. That is really the whole reason this model was chosen over something more flexible.

4.2 Preparing the Data

4.2.1 Target Encoding

`credit_risk` was encoded so that $y = 1$ means bad and $y = 0$ means good, since from a lender's point of view, what actually matters is catching the applicants likely to default.

4.2.2 One-Hot Encoding

The 13 categorical features were one-hot encoded, dropping one category per feature as the reference level (otherwise you run into the dummy variable trap). This turned the 13 categorical columns into 41 dummy columns, which together with the 7 numerical features gives a design matrix of 48 predictors in total.

4.2.3 Train-Test Split

The data was split 80/20 into training ($n = 800$) and test ($n = 200$) sets, using stratified sampling so the 70/30 class split is preserved exactly in both halves.

4.2.4 Scaling

The seven numerical features were standardized (mean 0, standard deviation 1) using a scaler fit only on the training data, mainly to help the optimizer converge smoothly and to make the coefficients a bit easier to compare against each other. The dummy variables were left as 0/1 and not scaled.

4.3 Checking Multicollinearity Again on the Full Matrix

Once all 48 predictors were assembled, VIF was recalculated on the full training matrix. The highest values turned up among dummies belonging to the same parent feature (for example, `housing_own` and the `credit_history` dummies sit around 6 to 6.5), which is just what happens

mechanically when categories are mutually exclusive – it is not a sign of a real problem across different features. Nothing crossed the usual warning level of 10, so all 48 predictors were kept going into the modeling stage.

5 Full Logistic Regression Model

The full model, all 48 predictors plus an intercept, was fit using `statsmodels.Logit` on the training set. It converged without issue, with a log-likelihood of -356.53 against a null log-likelihood of -488.69 (likelihood ratio test against the null model gives $p < 10^{-30}$), and a McFadden’s pseudo R^2 of 0.270, which for a logistic model of this kind is considered quite good.

Out of the 48 predictors, only **12** came out significant at $\alpha = 0.05$ once everything else was held constant (Table 6). This is actually a fairly big departure from what the univariate tests in Section 3.6 suggested – several **purpose** categories, for instance, looked informative on their own but stopped being significant once stronger predictors like **checking_account_status**, **credit_history**, **duration_months**, and **credit_amount** were already in the model. In other words, a lot of what looked like a real effect in the univariate analysis was probably just correlated with these stronger variables rather than contributing anything independent.

Table 6: Statistically significant predictors in the full model ($\alpha = 0.05$), sorted by p -value.

Variable	Coefficient	Odds Ratio	p -value
checking_account_status_no checking account	-1.218	0.296	<0.0001
installment_rate_pct	0.406	1.501	0.0002
other_installment_plans_none	-0.888	0.411	0.0010
housing_rent	1.531	4.624	0.0071
personal_status_sex_male: single	-0.612	0.542	0.0096
duration_months	0.333	1.395	0.0107
credit_history_critical account/other credits	-1.198	0.302	0.0134
property_unknown/no property	1.177	3.245	0.0158
credit_amount	0.355	1.426	0.0169
purpose_education	1.193	3.296	0.0218
foreign_worker_yes	1.466	4.331	0.0375
savings_account_unknown/no savings account	-0.806	0.447	0.0379

6 Reduced Model via Backward Elimination

To find a smaller set of variables that is still statistically defensible, I ran backward elimination on the full model: at each step, drop the single least significant variable and refit, and keep going until everything left is significant at $\alpha = 0.05$. This took **31** steps and left **17** predictors standing (Table 7). The full step-by-step log is in Appendix A.

One thing I did not expect going in: the 17 variables that survive elimination are not simply the 12 that were significant in the full model. Five variables (**purpose_car** (new), **checking_account_status** < 0 DM, **savings_account** >= 1000 DM, **employment_since** 4-7 years, and **housing_own**) only became significant after other, less useful variables were removed. This makes sense once you think about it – whether a variable looks significant depends on what else is in the model with it, not just on the variable itself – but it is still a good reminder that significance from a single full-model fit should not be treated as the final word.

Table 7: Reduced model: 17 retained predictors, sorted by p -value.

Variable	Coefficient	Odds Ratio	p -value
checking_account_status_no checking account	-1.213	0.297	<0.0001
savings_account_unknown/no savings account	-1.110	0.330	0.0001
credit_history_critical account/other credits	-0.821	0.440	0.0003
other_installment_plans_none	-0.787	0.455	0.0003
purpose_car (new)	0.743	2.102	0.0005
installment_rate_pct	0.355	1.426	0.0005
housing_rent	1.752	5.765	0.0008
purpose_education	1.292	3.641	0.0015
personal_status_sex_male: single	-0.592	0.553	0.0025
duration_months	0.363	1.437	0.0027
property_unknown/no property	1.238	3.447	0.0047
checking_account_status_< 0 DM	0.546	1.727	0.0101
savings_account_>= 1000 DM	-1.400	0.247	0.0116
employment_since_4-7 years	-0.665	0.514	0.0118
housing_own	1.179	3.250	0.0179
credit_amount	0.285	1.330	0.0252
foreign_worker_yes	1.511	4.530	0.0300

7 Full Model vs. Reduced Model: Is the Smaller Model Good Enough?

Since the reduced model is nested inside the full model (every variable it keeps is also in the full model), it is possible to test directly whether dropping those 31 variables actually cost anything, using a **Likelihood Ratio Test**:

$$LR = -2(\ell_{\text{reduced}} - \ell_{\text{full}}) \sim \chi^2_{df}, \quad (2)$$

where ℓ is the log-likelihood and df is the difference in the number of parameters between the two models. The numbers are summarized in Table 8.

Table 8: Full model vs. reduced model: fit statistics and the Likelihood Ratio Test.

Metric	Full Model	Reduced Model
Number of predictors	48	17
Log-Likelihood	-356.53	-371.68
AIC	811.06	779.37
BIC	1040.60	863.69
McFadden Pseudo R^2	0.2704	0.2394

The test gives $LR = 30.31$ with $df = 31$ and $p = 0.50$. Since that p -value is nowhere near 0.05, we fail to reject the null hypothesis that the 31 dropped variables have zero effect jointly – meaning there is no real statistical evidence that keeping them in the model was worth it. AIC and BIC agree with this: both are lower for the reduced model, and since both penalize extra parameters, that is a vote for the smaller model rather than the bigger one. So on paper, at least, the reduced model is the better choice between the two.

8 How Well Do They Actually Predict?

Being statistically significant is one thing; actually predicting well on new applicants is a separate question, and that is what this section checks. Both models were run on the test set ($n = 200$), which was not touched during fitting. Since the classes are imbalanced, I am reporting precision, recall, F1-score, and ROC-AUC alongside accuracy (Table 9), not just accuracy on its own.

Table 9: Test-set predictive performance at the default 0.5 classification threshold.

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Full Model	0.780	0.643	0.600	0.621	0.799
Reduced Model	0.760	0.630	0.483	0.547	0.787

Looking closely at this table, the ROC-AUC values are pretty close (0.799 vs. 0.787, a gap of about 0.01), so the reduced model is nearly as good at *ranking* applicants by risk across all thresholds. But at the fixed 0.5 cutoff, the reduced model's recall on bad-risk applicants is noticeably worse (0.48 vs. 0.60), and I do not want to brush that under the rug just because the LRT in Section 7 came out in the reduced model's favor – it is a real difference on this test set, not a rounding error. That said, there are only 60 bad-risk applicants in the test set, so a handful of cases flipping from caught to missed shifts recall by a fair amount, and Section 9 shows that this gap actually shrinks a lot once both models get to use their own best threshold instead of both being stuck at 0.5.

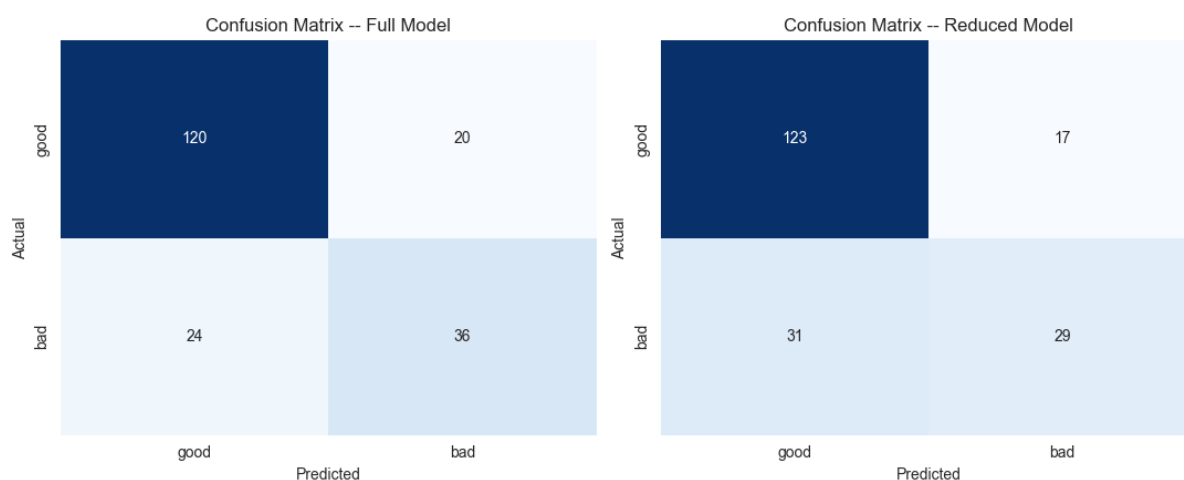


Figure 6: Confusion matrices for the full model (left) and reduced model (right) at the default 0.5 threshold.

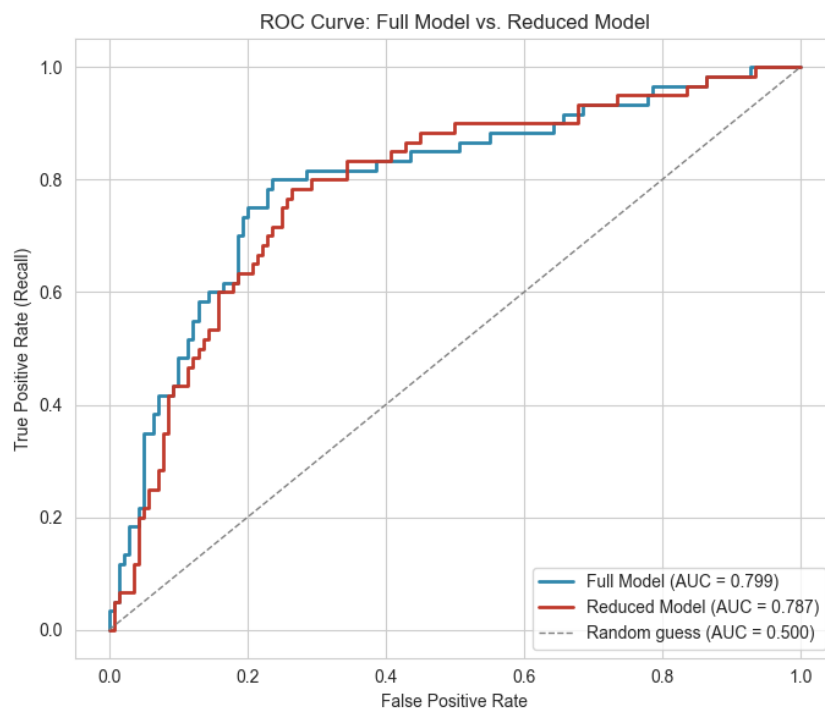


Figure 7: ROC curves for the full and reduced models on the test set.

9 Choosing a Threshold That Reflects the Real Cost

Everything up to this point used the standard 0.5 cutoff, which treats a false positive (calling a good applicant bad) and a false negative (missing a bad applicant) as equally bad. But the dataset's own cost matrix (Table 2) says a false negative is five times worse, so using 0.5 here does not really make sense. Instead, I searched across thresholds to find the one that minimizes total expected cost on the test set (Figure 8).

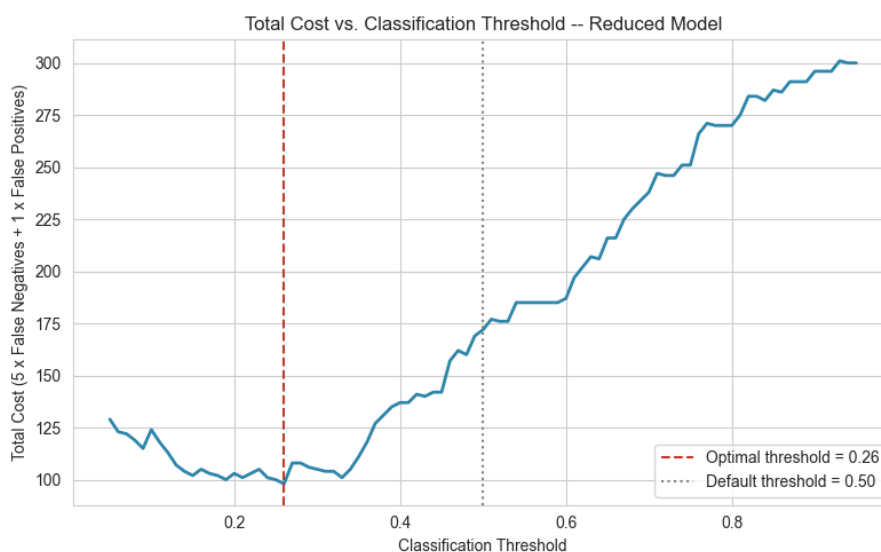


Figure 8: Total expected cost vs. classification threshold, reduced model.

For the reduced model, the best threshold turned out to be **0.26** (total cost 98, compared to

172 at 0.5 – a drop of about 43%). For the full model, the best threshold was **0.33** (total cost 94). What I find useful about this is that, once each model gets to use its own best threshold, the gap between them is only 4 cost units (94 vs. 98) – much smaller than it looked in Section 8. So a good chunk of the recall difference reported earlier was really just an artifact of judging both models at a threshold neither of them was tuned for.

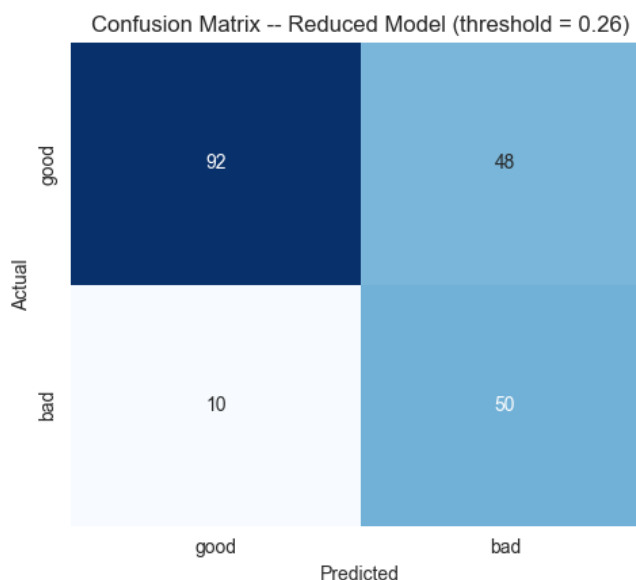


Figure 9: Confusion matrix for the reduced model at its cost-optimal threshold (0.26).

10 Final Model and What It Means

10.1 Which Model Should Actually Be Used

Putting everything from Sections 5 through 9 together, the evidence mostly points the same way but not completely. The Likelihood Ratio Test, AIC, and BIC all favor the reduced model, and its ROC-AUC is almost identical to the full model's. At a fixed threshold, the full model does keep a small edge in recall, but that edge gets much smaller once both models use their own cost-optimal threshold. Given all this, I am going with the **reduced model** as the final model for this report, mostly because the statistical tests support it and it is a lot easier to explain with 17 predictors instead of 48 – which matters if you actually have to justify a credit decision to someone. If recall was the only thing that mattered, keeping the full model would also be a reasonable call. Either way, the threshold from Section 9 should be used instead of the default 0.5.

10.2 Reading the Coefficients

Every row in Table 7 can be read as a statement about odds: for a one-standard-deviation change in a numerical predictor, or for belonging to a given category instead of the reference one, the odds of bad credit risk get multiplied by that odds ratio, with everything else held fixed. A few of the more interesting ones:

- Having **no checking account** is linked to a 70% *lower* odds of bad credit risk (OR = 0.30) compared to the reference category – the same counter-intuitive pattern flagged back in

Section 3.4.

- An **unknown/no savings account** is linked to a 67% lower odds of bad risk ($OR = 0.33$), while savings of 1,000 DM or more is linked to a 75% lower odds ($OR = 0.25$) – both compared to their own reference category, so the two numbers are not directly comparable to each other.
- **Renting** rather than owning is linked to almost a 5.8 times increase in bad-risk odds ($OR = 5.77$), the single largest effect in the model.
- A one-standard-deviation increase in **loan duration** or **credit amount** is linked to a 44% and 33% increase in bad-risk odds respectively, which lines up with the bivariate pattern seen back in Section 3.6.
- **Foreign worker** status shows a 4.5 times increase in bad-risk odds, but this is based on only 37 applicants out of 1,000, so I would not put too much weight on this particular number.

This is really the advantage of using Logistic Regression for this kind of problem: every one of these statements comes with an actual p -value behind it, not just a ranking of "important" features with no sense of how confident we should be in that ranking.

11 Conclusion and Limitations

11.1 Main Takeaways

This report set out to use Logistic Regression to figure out which applicant characteristics actually predict credit risk, and to end up with a model that is both statistically sound and reasonably easy to explain. `checking_account_status`, `credit_history`, `savings_account`, loan duration, credit amount, and housing situation came out as the most consistent predictors, holding up through both the univariate tests and the backward elimination process. The reduced model (17 predictors) was not statistically distinguishable from the full model (Likelihood Ratio Test, $p = 0.50$), and given how much simpler it is to explain, it was chosen as the final model – while still being upfront that it gives up a bit of recall at the default threshold compared to the full model. Once the threshold was adjusted to reflect the dataset's actual 5:1 cost ratio, that trade-off mostly evened out, and the model became noticeably more useful for an actual lending decision than it would be at the default 0.5 cutoff.

11.2 Limitations

- The dataset is fairly small ($n = 1,000$) and somewhat old, so the patterns found here might not hold up as strongly for a different, more recent population of applicants.
- A few categorical features have very few observations in some categories (`foreign_worker: no`, for instance, has only 37), which means the coefficients tied to those categories come with a lot of uncertainty.
- Only one train-test split was used here, and with just 60 bad-risk cases in the test set, metrics like recall can move around a fair bit just from sampling. Cross-validation would be the obvious next step to check how stable these results really are.

- Logistic Regression assumes the predictors relate to the log-odds linearly, and that was not formally checked here (a Box–Tidwell test would be one way to do that).
- The 5:1 cost ratio used in Section 9 comes straight from the dataset documentation. A real bank would presumably want to calculate this from its own default and loan-value data rather than borrow someone else’s number.

Appendix A: Full Backward Elimination Log

The table below lists every variable removed during backward elimination (Section 6), in the order it was removed, along with the p -value it had at that point.

Table 10: Backward elimination log (31 steps).

Step	Removed Variable	p -value
1	purpose_domestic appliances	0.9933
2	present_residence_since	0.9066
3	purpose_radio/television	0.8520
4	property_car or other	0.8430
5	employment_since_ $\geq$ 7 years	0.8369
6	personal_status_sex_male: married/widowed	0.7926
7	savings_account_500-1000 DM	0.7195
8	age_years	0.6542
9	job_unskilled resident	0.6463
10	job_skilled employee/official	0.7197
11	credit_history_no credits taken/all paid duly	0.5837
12	purpose_retraining	0.4860
13	purpose_others	0.4681
14	job_unemployed/unskilled non-resident	0.4178
15	other_debtors_guarantors_none	0.4219
16	employment_since_unemployed	0.3978
17	property_real estate	0.3816
18	checking_account_status_ $\geq$ 200 DM	0.3578
19	existing_credits_count	0.3429
20	personal_status_sex_male: divorced/separated	0.2910
21	credit_history_delay in paying off in the past	0.2788
22	credit_history_existing credits paid duly till now	0.2496
23	other_installment_plans_stores	0.2461
24	purpose_furniture/equipment	0.2452
25	savings_account_ $<$ 100 DM	0.2100
26	other_debtors_guarantors_guarantor	0.1779
27	num_dependents	0.1831
28	purpose_repairs	0.1432
29	employment_since_ $<$ 1 year	0.1121
30	purpose_car (used)	0.0628
31	telephone_yes, registered	0.0566

Appendix B: Data and Tools

Dataset citation: Hofmann, H. (1994). *Statlog (German Credit Data)*. UCI Machine Learning Repository.

Everything here was done in Python 3.11: `pandas` and `numpy` for handling the data, `matplotlib` and `seaborn` for the plots, `scipy.stats` for the hypothesis tests, `statsmodels` for the actual Logistic Regression fitting, and `scikit-learn` for splitting, scaling, and the predictive evaluation metrics. The two notebooks behind this report, `01_Exploratory_Data_Analysis.ipynb` and `02_Logistic_Regression_Modeling.ipynb`, contain the full working if anyone wants to check or rerun any of this.